

UNIVERSIDADE REGIONAL DE BLUMENAU - FURB
CIÊNCIAS DA COMPUTAÇÃO

PEDRO BOSINI FREITAG, SAMUEL CANDIDO E VITOR DA SILVA

ARQUITETURA DE UM SISTEMA DE MENSAGERIA DE E-COMMERCE:
processamento assíncrono de pedidos com RabbitMQ

BLUMENAU
2026

1 DESCRIÇÃO

Este documento descreve um sistema de e-commerce que processa pedidos de forma assíncrona que utilizam do RabbitMQ para a troca de mensagens.

A aplicação simula uma loja com alta demanda de pedidos. Cada pedido passa por 3 etapas de processamento: reserva de itens do estoque, processamento do pagamento e agendamento da entrega.

O processamento de pedidos precisa ser assíncrono para que a API continue rápida e disponível mesmo quando algum serviço integrado estiver lento ou fora do ar.

1.1 Necessidade

Em uma abordagem síncrona, a API de Pedidos chamaria diretamente os serviços de estoque, pagamento e logística e só responderia ao cliente ao final de todas as chamadas. Isso torna a resposta lenta, pois o tempo total é a soma dos tempos de cada serviço. Além disso, a falha de qualquer serviço se propaga até a API, gerando falhas em cascata.

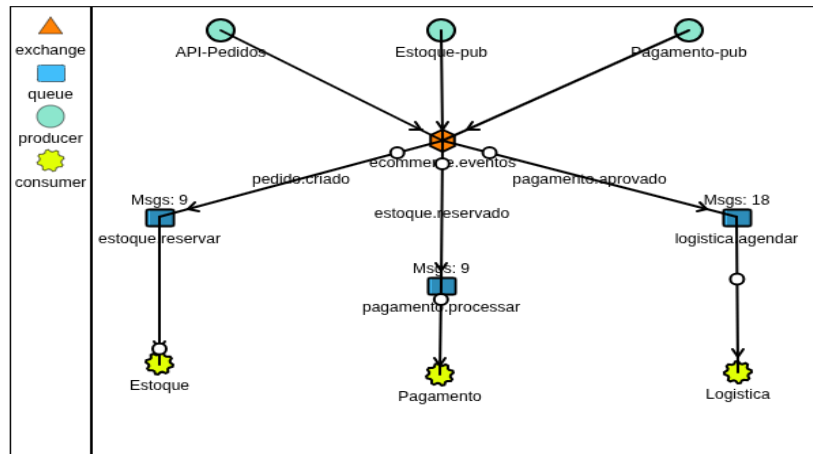
1.3 Justificativa da mensageria

A mensageria permite desacoplar o processamento do pedido da resposta ao cliente. A API apenas registra o pedido e publica um evento, respondendo imediatamente. As demais etapas são executadas de forma assíncrona pelos serviços responsáveis. Com isso, o sistema ganha escalabilidade, pois cada serviço pode ser ampliado conforme a demanda, e confiabilidade, pois as mensagens ficam armazenadas no broker até serem processadas.

2 ARQUITETURA DA SOLUÇÃO

A arquitetura é formada por quatro papéis: produtor, broker, consumidor e Dead Letter Queue (DLQ). Figura 1 – Mensageria da solução

Mensageria da solução



2.1 Produtor

O produtor é o componente que publica mensagens na exchange. O ponto de entrada é o controller da API de Pedidos, que publica a mensagem do pedido informando uma routing key. Os demais serviços também atuam como produtores, pois publicam o evento resultante da sua etapa.

2.2 Broker

O broker é o RabbitMQ, composto pela exchange e pelas filas. A exchange recebe as mensagens e as encaminha para a fila correspondente de acordo com a routing key de cada evento.

2.3 Consumidor

Cada serviço consome a sua própria fila e executa apenas o seu passo do fluxo: reservar estoque, cobrar ou agendar a entrega. Ao terminar, publica o próximo evento, dando continuidade ao processo.

2.4 Dead Letter Queue - DLQ

A DLQ recebe as mensagens que não puderam ser processadas por falha técnica ou que expiraram. Dessa forma, nenhuma mensagem é descartada e os pedidos com problema podem ser inspecionados e reprocessados manualmente.

3 SERVIÇOS DA SOLUÇÃO

A solução possui quatro serviços independentes, que se comunicam apenas por mensagens e não fazem chamadas diretas entre si.

3.1 API de Pedidos

É o produtor inicial. Recebe o pedido do cliente pela API, persiste o pedido e publica o evento `pedido.criado` na exchange.

3.2 Serviço de Estoque

Consume o evento `pedido.criado` e tenta reservar os itens do pedido. Em caso de sucesso, publica `estoque.reservado`;

3.3 Serviço de Pagamento

Consume o evento `estoque.reservado` e processa a cobrança. Publica `pagamento.aprovado` quando a cobrança é concluída.

3.4 Serviço de Logística

Consume o evento `pagamento.aprovado`, agenda a entrega do pedido.

4 FILAS, EVENTOS E ROTEAMENTO

A solução utiliza uma única exchange central, e a routing key de cada evento define a fila de destino. Quadro 1 – Roteamento de eventos

Roteamento de eventos

Produtor	Evento (routing key)	Fila	Consumidor
API de Pedidos	pedido.criado	estoque.reservar	Serviço de Estoque
Serviço de Estoque	estoque.reservado	pagamento.processar	Serviço de Pagamento
Serviço de Pagamento	pagamento.aprovado	logistica.agendar	Serviço de Logística
Qualquer serviço	falha técnica (nack)	ecommerce.dlq	Inspeção manual

4.1 Exchange e DLX

A exchange central é a `ecommerce.eventos`, do tipo `topic` e durável. O tipo `topic` foi escolhido porque permite rotear as mensagens pelo nome do evento e facilita a inclusão de novos consumidores no futuro sem alterar os produtores. As filas principais apontam para a Dead Letter Exchange `ecommerce.dlx`, que encaminha as mensagens com falha para a fila `ecommerce.dlq`.

4.2 Formato da mensagem

As mensagens são trafegadas em formato JSON, com os seguintes campos:

- a) `pedidoId`: identificador do pedido;
- b) `clienteId`: identificador do cliente;
- c) `itens`: lista de itens do pedido;
- d) `valorTotal`: valor total do pedido;
- e) `mensagemId`: identificador único da mensagem, usado para evitar processamento duplicado;
- f) `timestamp`: data e hora de criação da mensagem.

5 REGRAS DE TIMEOUT E DLQ

Somente falhas técnicas e mensagens expiradas são encaminhadas para a DLQ. A população da DLQ e da DLX com eventos de erro ocorrerá de forma aleatória, sem seguir uma regra de negócio, apenas para fins de simulação.

5.1 Falha técnica

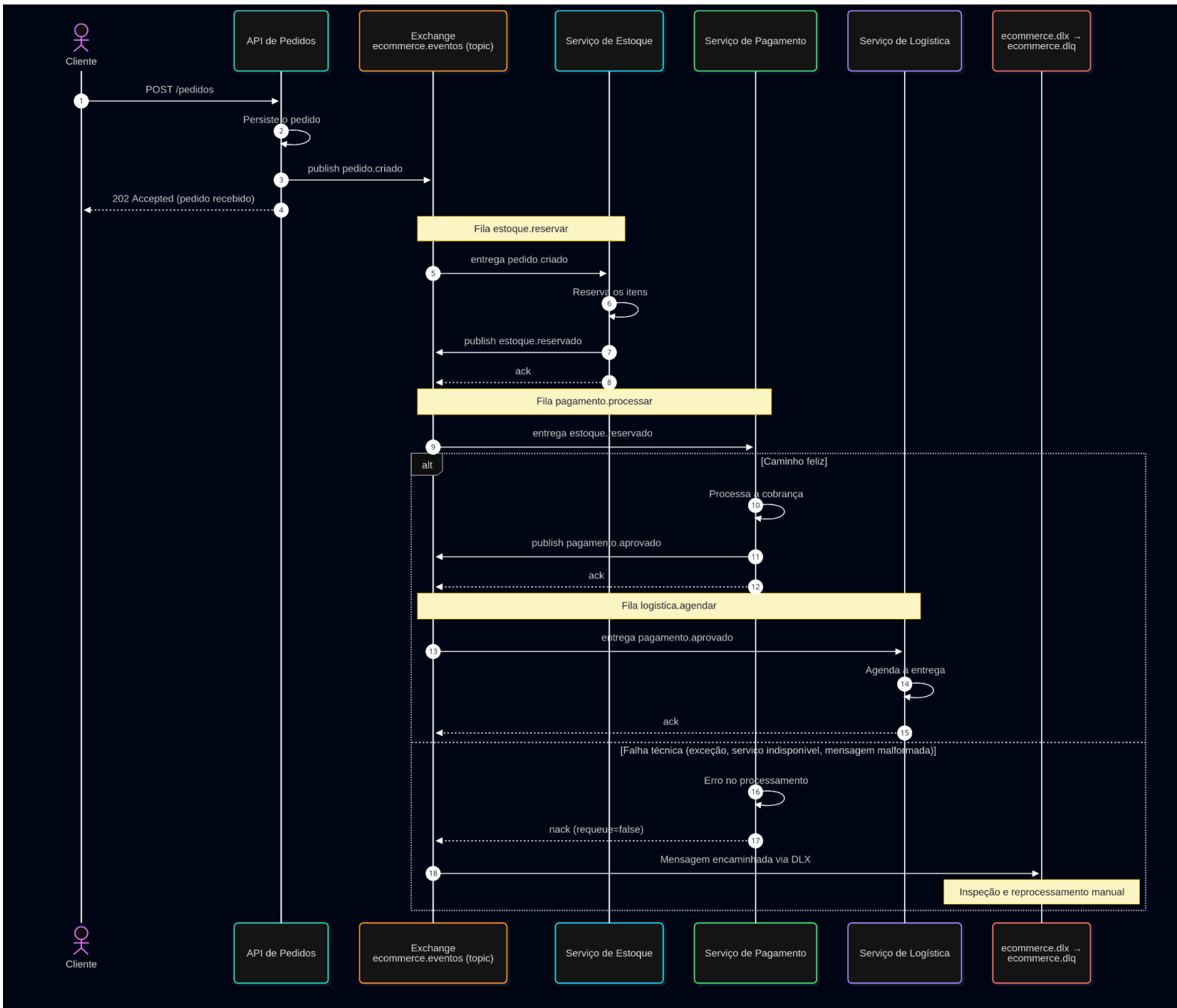
São consideradas falhas técnicas as exceções durante o processamento, a indisponibilidade de um serviço necessário e as mensagens malformadas. Nesses casos, o consumidor executa `nack` com `requeue=false` e o RabbitMQ encaminha a mensagem para a DLQ por meio da DLX.

5.2 Tempo de expiração (TTL)

As filas principais são configuradas com `x-message-ttl` de 10 minutos. Uma mensagem que não for consumida dentro desse prazo expira e é redirecionada. Após três tentativas sem sucesso, a mensagem é enviada em definitivo para a DLQ.

6 FLUXO PRINCIPAL

Figura 2 – Diagrama de sequência do processamento de pedidos



As etapas do fluxo são:

- a) o cliente realiza o pedido;
- b) a API recebe a requisição e persiste o pedido;
- c) a API publica o evento **pedido.criado** na exchange;
- d) a exchange roteia a mensagem pela routing key até a fila **estoque.reservar**;
- e) o Serviço de Estoque consome o evento **pedido.criado**;
- f) o Serviço de Estoque reserva os itens e publica **estoque.reservado**;
- g) o Serviço de Pagamento consome o evento, realiza a cobrança e publica **pagamento.aprovado**;
- h) o Serviço de Logística consome o evento e agenda a entrega;
- i) o pedido é confirmado ao cliente;
- j) qualquer falha técnica durante o fluxo é direcionada para a DLQ.

7 ESTRATÉGIAS E CONFIGURAÇÃO

A configuração do RabbitMQ foi definida para sustentar três requisitos: confiabilidade, escalabilidade e tolerância a falhas.

7.1 Confiabilidade

A exchange e as filas são declaradas como duráveis e as mensagens são publicadas como persistentes (`delivery_mode=2`), o que permite que sobrevivam a uma reinicialização do broker. O consumidor utiliza confirmação manual (`ack`) e só confirma a mensagem após concluir o processamento com sucesso. Assim, se uma instância do serviço cair durante o processamento, a mensagem não confirmada volta a ficar disponível na fila e nenhum pedido é perdido.

7.2 Escalabilidade

Várias instâncias de um mesmo serviço podem consumir a mesma fila ao mesmo tempo. O `prefetch`, configurado por meio de `basic.qos`, limita a quantidade de mensagens não confirmadas entregues a cada instância, distribuindo a carga de forma equilibrada entre elas. Em períodos de pico, basta aumentar o número de instâncias do serviço mais sobrecarregado.

7.3 Tolerância a falhas

Uma falha técnica gera `nack` com `requeue=false` e a mensagem é encaminhada para a DLQ, sem bloquear o processamento das demais mensagens da fila. Como a API apenas publica o evento, uma falha em um serviço posterior não impede o recebimento de novos pedidos.

8 BENEFÍCIOS DA ARQUITETURA

O desacoplamento do processamento por meio da mensageria traz três benefícios principais:

- a) escalabilidade: múltiplas instâncias de um serviço podem consumir a mesma fila em paralelo, absorvendo picos de pedidos;
- b) confiabilidade: a mensagem permanece na fila até ser confirmada, garantindo que nenhum pedido seja perdido;
- c) tolerância a falhas: falhas no processamento não derrubam a API, e as mensagens problemáticas são isoladas na DLQ.

9 CONSIDERAÇÕES FINAIS

O uso do RabbitMQ desacopla o processamento do pedido da resposta da API, tornando o sistema mais responsivo para o cliente. A divisão em serviços independentes, ligados por uma exchange central, permite escalar produtores e consumidores de forma independente, conforme a demanda de cada etapa.

A DLQ garante que as falhas de processamento sejam isoladas e tratadas sem perda de pedidos, enquanto as demais mensagens seguem o fluxo sem bloqueio. O resultado é um comércio eletrônico mais confiável, resiliente e preparado para o crescimento.